Biostat 212a Homework 5

Due Mar 16, 2024 @ 11:59PM

Nathaniel Boyle and 006525153

2025-03-13

Table of contents

0.1 ISL Exercise 9.7.1 (10pts)

0.1.1 1. Hyperplanes in two dimensions.

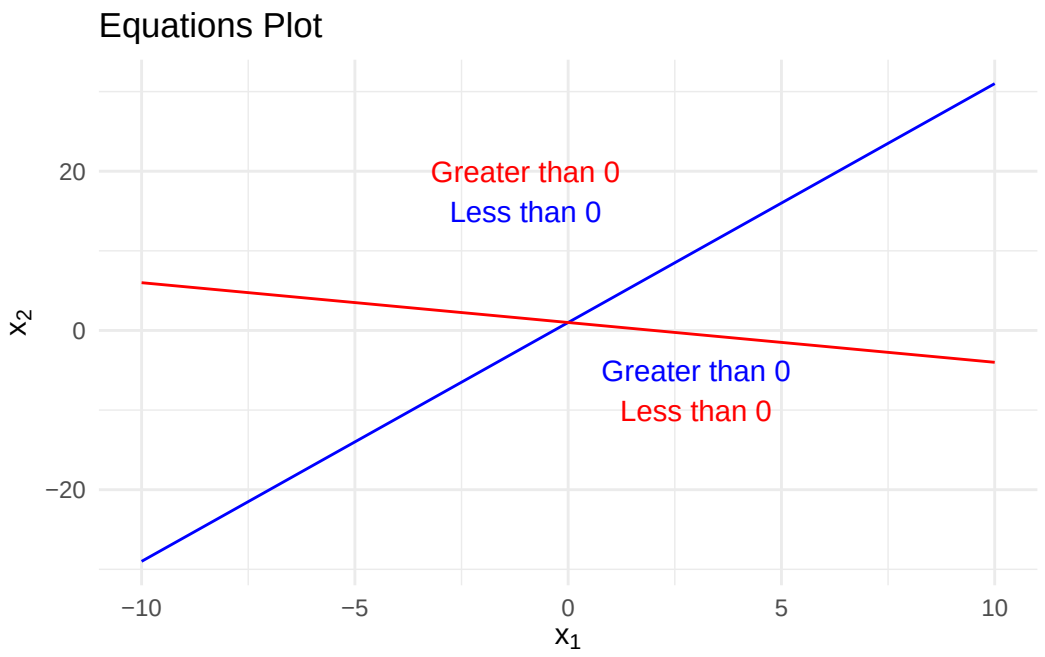
0.1.1.1 part a. and part b.

Sketch the hyperplane $1+3X_1-X_2=0$. Indicate the set of points for which $1+3X1-X2>0$ as well as the set of points for which $1+3X_1-X_2<0$.

On the same plot, sketch the hyperplane $-2+X_1+2X_2=0$ Indicate the set of points for which $-2+X_1+2X_2>0$ as well as the set of points for which $-2+X_1+2X_2<0$

```
library(ggplot2)

# Define the data for the two equations
x1 <- -10:10
x2 <- 1 + 3 * x1
```

```r
x3 <- 1 - x1 / 2

# Create a data frame for ggplot
df <- data.frame(x1 = x1, x2 = x2, x3 = x3)

# Plot the graph using ggplot2
ggplot(df, aes(x = x1)) +
  # Plot the first line (blue)
  geom_line(aes(y = x2), color = "blue") +

  # Annotate text for the blue line
  annotate("text", x = 3, y = -5, label = "Greater than 0", color = "blue") +
  annotate("text", x = -1, y = 15, label = "Less than 0", color = "blue") +

  # Plot the second line (red)
  geom_line(aes(y = x3), color = "red") +

  # Annotate text for the red line
  annotate("text", x = 3, y = -10, label = "Less than 0", color = "red") +
  annotate("text", x = -1, y = 20, label = "Greater than 0", color = "red") +

  # Add the legend with LaTeX formatting
  scale_color_manual(
    values = c("blue", "red"),
    labels = c(
      expression(1 + 3 * x[1] - x[2] == 0),
      expression(-2 + x[1] + 2 * x[2] == 0)
    )
  ) +

  # Add labels and title
  labs(
    title = "Equations Plot",
    x = expression(x[1]),
    y = expression(x[2])
  ) +

  # Set theme for better appearance
  theme_minimal() +
  theme(legend.position = "right")
```



0.2 ISL Exercise 9.7.2 (10pts)

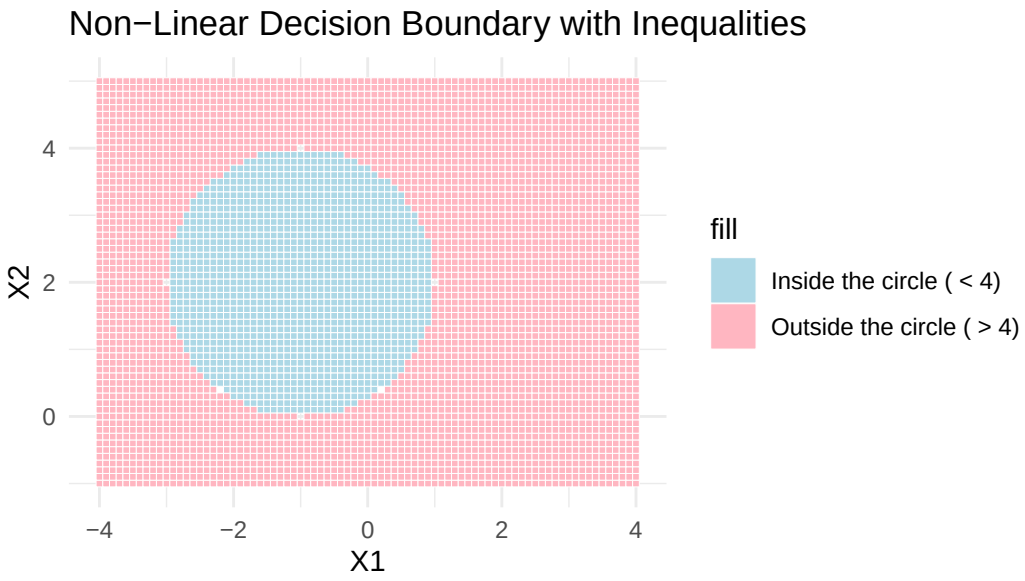
We have seen that in $p = 2$ dimensions, a linear decision boundary takes the form $\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$. We now investigate a non-linear decision boundary.

0.2.1 2.a and 2.b

Sketch the curve $(1+X_1)^2+(2-X2)^2 = 4$. On your sketch, indicate the set of points for which $(1 + X1)^2 + (2 - X2)^2 > 4$, as well as the set of points for which $(1 + X1)^2 + (2 - X2)^2 \leq 4$

```
# Load ggplot2 library
library(ggplot2)

# Define the equation of the circle (1 + X1)^2 + (2 - X2)^2 = 4
# The center is (1, 2) and the radius is 2

radius <- 2
center_x <- 1
center_y <- 2

# Generate points to plot the circle
```

```r
theta <- seq(0, 2*pi, length.out = 100)  # Angle sequence
x1 <- center_x + radius * cos(theta)  # X1 coordinates
x2 <- center_y + radius * sin(theta)  # X2 coordinates

# Create a data frame for ggplot
circle_data <- data.frame(X1 = x1, X2 = x2)

# Create a grid of points to evaluate the inequality
grid_x1 <- seq(-4, 4, by = 0.1)
grid_x2 <- seq(-1, 5, by = 0.1)
grid <- expand.grid(X1 = grid_x1, X2 = grid_x2)

# Evaluate the inequality (1 + X1)^2 + (2 - X2)^2 at each point
grid$distance_squared <- (1 + grid$X1)^2 + (2 - grid$X2)^2

# Plot the circle using ggplot
ggplot() +
  # Shading for (1 + X1)^2 + (2 - X2)^2 < 4 (inside the circle)
  geom_tile(data = subset(grid, distance_squared < 4),
            aes(x = X1, y = X2, fill = "Inside the circle ( < 4)"),
            color = "white", width = 0.1, height = 0.1) +

  # Shading for (1 + X1)^2 + (2 - X2)^2 > 4 (outside the circle)
  geom_tile(data = subset(grid, distance_squared > 4),
            aes(x = X1, y = X2, fill = "Outside the circle ( > 4)"),
            color = "white", width = 0.1, height = 0.1) +


  # Add labels and title
  labs(x = "X1", y = "X2",
       title = "Non-Linear Decision Boundary with Inequalities") +

  # Theme and legend customization
  theme_minimal() +
  scale_fill_manual(
    values = c("Inside the circle ( < 4)" = "lightblue",
               "Outside the circle ( > 4)" = "lightpink")) +

  coord_fixed(ratio = 1) +  # Ensure equal aspect ratio
  theme(legend.position = "right")  # Place legend to the right
```



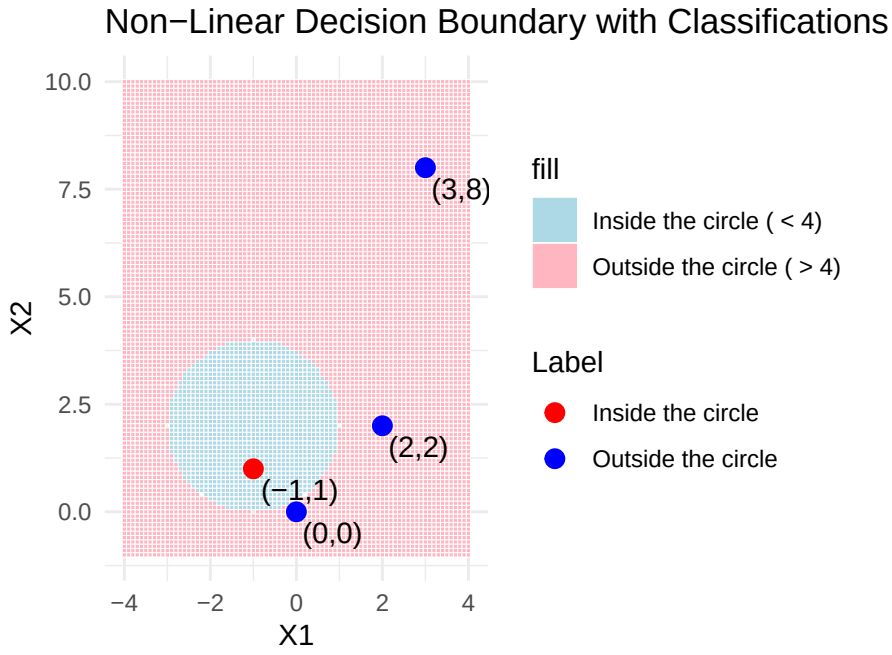
0.2.2 2.c

Suppose that a classier assigns an observation to the blue class if $(1 + X1)2 + (2 - X2)2 > 4$, and to the red class otherwise. To what class is the observation $(0, 0)$ classified? $(-1, 1)$? $(2, 2)$? $(3, 8)$?

```
# Load ggplot2 library
library(ggplot2)

# Define the equation of the circle (1 + X1)^2 + (2 - X2)^2 = 4
# The center is (1, 2) and the radius is 2

radius <- 2
center_x <- 1
center_y <- 2

# Generate points to plot the circle
theta <- seq(0, 2*pi, length.out = 100)  # Angle sequence
x1 <- center_x + radius * cos(theta)  # X1 coordinates
x2 <- center_y + radius * sin(theta)  # X2 coordinates

# Create a data frame for ggplot
```

```r
circle_data <- data.frame(X1 = x1, X2 = x2)

# Create a grid of points to evaluate the inequality
grid_x1 <- seq(-4, 4, by = 0.1)
grid_x2 <- seq(-1, 10, by = 0.1)
grid <- expand.grid(X1 = grid_x1, X2 = grid_x2)

# Evaluate the inequality (1 + X1)^2 + (2 - X2)^2 at each point
grid$distance_squared <- (1 + grid$X1)^2 + (2 - grid$X2)^2

# Define the observations to classify
observations <- data.frame(
  X1 = c(0, -1, 2, 3),
  X2 = c(0, 1, 2, 8),
  Label = c("Outside the circle", "Inside the circle",
            "Outside the circle", "Outside the circle")
)

# Plot the circle using ggplot2
ggplot() +
  # Shading for (1 + X1)^2 + (2 - X2)^2 < 4 (inside the circle)
  geom_tile(data = subset(grid, distance_squared < 4),
            aes(x = X1, y = X2, fill = "Inside the circle ( < 4)"),
            color = "white", width = 0.1, height = 0.1) +

  # Shading for (1 + X1)^2 + (2 - X2)^2 > 4 (outside the circle)
  geom_tile(data = subset(grid, distance_squared > 4),
            aes(x = X1, y = X2, fill = "Outside the circle ( > 4)"),
            color = "white", width = 0.1, height = 0.1) +

  # Plot the observations with classifications
  geom_point(data = observations,
             aes(x = X1, y = X2, color = Label), size = 3) +

  # Add coordinates as text labels next to the points
  geom_text(data = observations,
            aes(x = X1, y = X2, label = paste("(", X1, ",", X2, ")", sep = "")),
            vjust = 1.5, hjust = -0.1, size = 4, color = "black") +

  # Add labels and title
  labs(x = "X1", y = "X2",
       title = "Non-Linear Decision Boundary with Classifications") +
```

```r
# Theme and legend customization
theme_minimal() +

scale_fill_manual(
  values = c("Inside the circle ( < 4)" = "lightblue",
             "Outside the circle ( > 4)" = "lightpink")) +
scale_color_manual(
  values = c("Outside the circle" = "blue", "Inside the circle" = "red")) +

coord_fixed(ratio = 1) +  # Ensure equal aspect ratio
theme(legend.position = "right")  # Place legend to the right
```



0.2.3 2.d

Argue that while the decision boundary in (c) is not linear in terms of X_1 and X_2, it is linear in terms of X_1, X_1^2 , X_2, andX_2^2.

The decision boundary in the given problem is represented by the equation:

$$(1 + X_1)^2 + (2 - X_2)^2 = 4$$

Although this represents a circle, if we transform the variables we can express this boundary in a linear form.

Expanding our squares to represent our boundary in terms of X_1, X_1^2 , X_2, and X_2^2:

$$(1 + 2X_1 + X_1^2) + (4 - 4X_2 + X_2^2) = 4$$

Simplify:

$$2X_1 - 4X_2 + X_1^2 + X_2^2 = -1$$

We can now transform our boundary terms into new, Linear terms of Z_n, where n is an integer that represents a new variable transformation for any X_n.

$$Z_1 = X_1$$
$$Z_2 = X_1^2$$
$$Z_3 = X_2$$
$$Z_4 = X_2^2$$

Now our decision boundary becomes:

$$2Z_1 - 4Z_3 + Z_2 + Z_4 = -1$$

This *is* a linear equation. Therefore our decision boundary is linear in terms of X_1, X_1^2 , X_2, andX_2^2 when we transform them to Z_n This is the core of what SVMs do with non-linear boundaries. Transforming the data into higher-dimensional feature space allows models to apply linear classifiers to non-linear equations.

0.3 Support vector machines (SVMs) on the `Carseats` data set (30pts)

Follow the machine learning workflow to train support vector classifier (same as SVM with linear kernel), SVM with polynomial kernel (tune the degree and regularization parameter C), and SVM with radial kernel (tune the scale parameter γ and regularization parameter C) for classifying `Sales<=8` versus `Sales>8`. Use the same seed as in your HW4 for the initial test/train split and compare the final test AUC and accuracy to those methods you tried in HW4.

```r
# Check if packages are installed and install them if not
if (!requireNamespace("ISLR", quietly = TRUE)) {
  install.packages("ISLR", repos = "https://cloud.r-project.org/")
}
if (!requireNamespace("e1071", quietly = TRUE)) {
  install.packages("e1071", repos = "https://cloud.r-project.org/")
}
if (!requireNamespace("caret", quietly = TRUE)) {
  install.packages("caret", repos = "https://cloud.r-project.org/")
}

# Load the libraries
library(ISLR)
```

```
Warning: package 'ISLR' was built under R version 4.4.3
```

```r
library(e1071)  # SVM implementation
```

```
Warning: package 'e1071' was built under R version 4.4.3
```

```r
library(caret)   # For model training and tuning
```

```
Warning: package 'caret' was built under R version 4.4.3
```

```r
# Load the carseats dataset
data("Carseats")

# Create a binary target variable: Sales <= 8 versus Sales > 8
Carseats$SalesBinary <- ifelse(Carseats$Sales <= 8, 0, 1)

# Create a training and testing split
set.seed(123)
trainIndex <- createDataPartition(Carseats$SalesBinary, p = 0.7, list = FALSE)
trainData <- Carseats[trainIndex, ]
testData <- Carseats[-trainIndex, ]

# Standardize the training and testing sets
preProcValues <- preProcess(trainData[, -c(1, 2, 11)], method = c("center",
                                                                  "scale"))
```

```r
trainDataScaled <- predict(preProcValues, trainData[, -c(1, 2, 11)])
testDataScaled <- predict(preProcValues, testData[, -c(1, 2, 11)])

# Add the target variable back
trainDataScaled$SalesBinary <- trainData$SalesBinary
testDataScaled$SalesBinary <- testData$SalesBinary

# Ensure that SalesBinary is a factor before making predictions
trainDataScaled$SalesBinary <- factor(trainDataScaled$SalesBinary,
                                      levels = c(0, 1))
testDataScaled$SalesBinary <- factor(testDataScaled$SalesBinary,
                                     levels = c(0, 1))

# SVM with linear kernel
svm_linear <- svm(SalesBinary ~ .,
                  data = trainDataScaled, kernel = "linear")

# Make predictions
pred_linear <- predict(svm_linear, testDataScaled)

# Convert the predicted values to factors (for linear kernel)
pred_linear_factor <- factor(pred_linear, levels = c(0, 1))

# Evaluate the model for linear kernel
confusionMatrix(pred_linear_factor, testDataScaled$SalesBinary)
```

```
Confusion Matrix and Statistics

          Reference
Prediction  0  1
         0 62 12
         1  9 37

               Accuracy : 0.825
                 95% CI : (0.745, 0.8883)
    No Information Rate : 0.5917
    P-Value [Acc > NIR] : 3.691e-08

                  Kappa : 0.6344

 Mcnemar's Test P-Value : 0.6625
```

```
            Sensitivity : 0.8732
            Specificity : 0.7551
         Pos Pred Value : 0.8378
         Neg Pred Value : 0.8043
             Prevalence : 0.5917
         Detection Rate : 0.5167
   Detection Prevalence : 0.6167
      Balanced Accuracy : 0.8142

       'Positive' Class : 0
```

```r
# Tune SVM with polynomial kernel
tune_poly <- tune(svm, SalesBinary ~ ., data = trainDataScaled,
                  kernel = "polynomial",
                  ranges = list(degree = 2:4, cost = 10^(-1:1)))

# Get the best model
svm_poly <- tune_poly$best.model

# Make predictions for the polynomial kernel
pred_poly <- predict(svm_poly, testDataScaled)

# Convert the predicted values to factors (for polynomial kernel)
pred_poly_factor <- factor(pred_poly, levels = c(0, 1))

# Evaluate the model for polynomial kernel
confusionMatrix(pred_poly_factor, testDataScaled$SalesBinary)
```

```
Confusion Matrix and Statistics

          Reference
Prediction  0  1
         0 58 13
         1 13 36

               Accuracy : 0.7833
                 95% CI : (0.6989, 0.8533)
    No Information Rate : 0.5917
    P-Value [Acc > NIR] : 7.182e-06

                  Kappa : 0.5516
```

```
 Mcnemar's Test P-Value : 1

            Sensitivity : 0.8169
            Specificity : 0.7347
         Pos Pred Value : 0.8169
         Neg Pred Value : 0.7347
             Prevalence : 0.5917
         Detection Rate : 0.4833
   Detection Prevalence : 0.5917
      Balanced Accuracy : 0.7758

       'Positive' Class : 0
```

```r
# Tune SVM with radial kernel
tune_radial <- tune(svm, SalesBinary ~ ., data = trainDataScaled,
                    kernel = "radial",
                    ranges = list(sigma = seq(0.01, 1, by = 0.1),
                                  cost = 10^(-1:1)))

# Get the best model
svm_radial <- tune_radial$best.model

# Make predictions for radial kernel
pred_radial <- predict(svm_radial, testDataScaled)

# Convert the predicted values to factors (for radial kernel)
pred_radial_factor <- factor(pred_radial, levels = c(0, 1))

# Ensure that SalesBinary in the test set is a factor with the same levels
testDataScaled$SalesBinary <- factor(testDataScaled$SalesBinary,
                                     levels = c(0, 1))

# Evaluate the model for radial kernel
confusionMatrix(pred_radial_factor, testDataScaled$SalesBinary)
```

```
Confusion Matrix and Statistics

          Reference
Prediction  0  1
         0 62 13
```

```
         1  9 36

               Accuracy : 0.8167
                 95% CI : (0.7357, 0.8814)
    No Information Rate : 0.5917
    P-Value [Acc > NIR] : 1.175e-07

                  Kappa : 0.6157

 Mcnemar's Test P-Value : 0.5224

            Sensitivity : 0.8732
            Specificity : 0.7347
         Pos Pred Value : 0.8267
         Neg Pred Value : 0.8000
             Prevalence : 0.5917
         Detection Rate : 0.5167
   Detection Prevalence : 0.6250
      Balanced Accuracy : 0.8040

       'Positive' Class : 0
```

```
# Compare the accuracy of each model
cat("Linear Kernel Accuracy: ",
    sum(pred_linear_factor == testDataScaled$SalesBinary) / length(
      pred_linear_factor), "\n")
```

```
Linear Kernel Accuracy:  0.825
```

```
cat("Polynomial Kernel Accuracy: ",
    sum(pred_poly_factor == testDataScaled$SalesBinary) / length(
      pred_poly_factor), "\n")
```

```
Polynomial Kernel Accuracy:  0.7833333
```

```
cat("Radial Kernel Accuracy: ",
    sum(pred_radial_factor == testDataScaled$SalesBinary) / length(
      pred_radial_factor), "\n")
```

```
Radial Kernel Accuracy:  0.8166667
```

Use the same seed as in your HW4 for the initial test/train split and compare the final test AUC and accuracy to those methods you tried in HW4.

```
# Load the pROC package for AUC calculation
if (!require("pROC", quietly = TRUE)) install.packages("pROC")
library(pROC)

# --- Linear Kernel ---
# Calculate accuracy for linear kernel
linear_accuracy <- sum(
  pred_linear_factor == testDataScaled$SalesBinary) / length(pred_linear_factor)

# Calculate AUC for linear kernel
roc_linear <- roc(testDataScaled$SalesBinary, as.numeric(pred_linear_factor))
linear_auc <- auc(roc_linear)

# --- Polynomial Kernel ---
# Calculate accuracy for polynomial kernel
poly_accuracy <- sum(
  pred_poly_factor == testDataScaled$SalesBinary) / length(pred_poly_factor)

# Calculate AUC for polynomial kernel
roc_poly <- roc(testDataScaled$SalesBinary, as.numeric(pred_poly_factor))
poly_auc <- auc(roc_poly)

# --- Radial Kernel ---
# Calculate accuracy for radial kernel
radial_accuracy <- sum(
  pred_radial_factor == testDataScaled$SalesBinary) / length(pred_radial_factor)

# Calculate AUC for radial kernel
roc_radial <- roc(testDataScaled$SalesBinary, as.numeric(pred_radial_factor))
radial_auc <- auc(roc_radial)

# --- Print the results ---
cat("Linear Kernel Accuracy: ", linear_accuracy, "\n")
```

```
Linear Kernel Accuracy:  0.825
```

```
cat("Linear Kernel AUC: ", linear_auc, "\n")
```

```
Linear Kernel AUC:  0.8141707
```

```
cat("Polynomial Kernel Accuracy: ", poly_accuracy, "\n")
```

```
Polynomial Kernel Accuracy:  0.7833333
```

```
cat("Polynomial Kernel AUC: ", poly_auc, "\n")
```

```
Polynomial Kernel AUC:  0.7757976
```

```
cat("Radial Kernel Accuracy: ", radial_accuracy, "\n")
```

```
Radial Kernel Accuracy:  0.8166667
```

```
cat("Radial Kernel AUC: ", radial_auc, "\n")
```

```
Radial Kernel AUC:  0.8039667
```

Decision Tree Statistics: Accuracy : 0.7342 ROC AUC: 0.809508

Random Forest Statistics: ROC AUC for Random Forest: 0.8134615 Accuracy for Random Forest: 0.8282828

XGBoost statistics: Accuracy : 0.8228 XGBoost ROC AUC: 0.9235372

The linear kernel model was the best performing model when compared to the other SVM kernel models, boasting the highest accuracy and ROC AUC value among the three used here. It also performed better than both the Decision Tree as well as the random forest models in terms of accuracy and ROC AUC.

The polynomial kernel was the worst performing SVM model and only performed marginally better in terms of accuracy and AUC than the decision tree.

The Radial Kernel performed slightly worse than the polynomial kernel and both decision tree and random forest models.

It seems that the best model was the XGBoost model out of all models tested, scoring the highest accuracy and ROC AUC value.

0.4 Bonus (10pts)

Let

$$f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p = \beta_0 + \beta^T X.$$

Then $f(X) = 0$ defines a hyperplane in $\mathbb{R}^p$. Show that $f(x)$ is proportional to the signed distance of a point x to the hyperplane $f(X) = 0$.